

Personalized Networking Assistant

Introduction

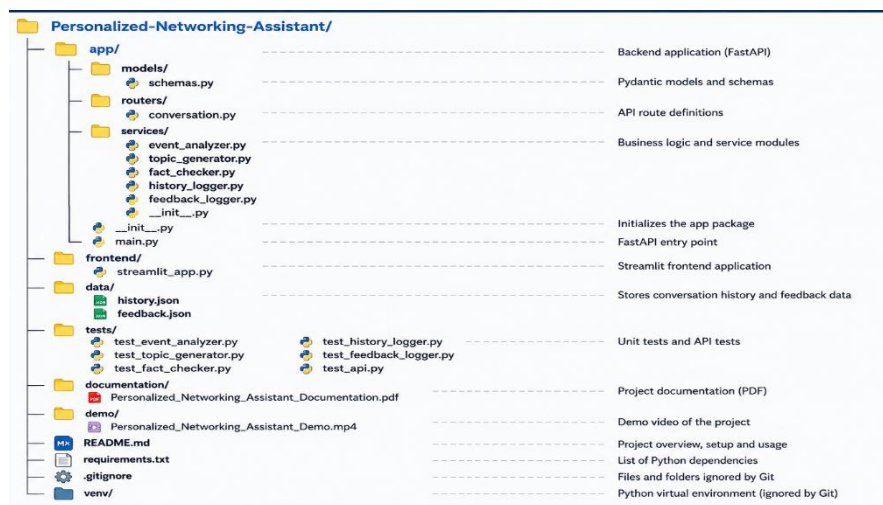
The **Personalized Networking Assistant** is an AI-powered web application developed to help participants build meaningful professional connections during conferences, seminars, workshops, hackathons, and technical events. Many attendees struggle to start conversations with strangers because they are unfamiliar with the event topics or lack confidence in initiating discussions. This project addresses that challenge by automatically analyzing an event description, identifying its major themes, and generating personalized conversation starters based on the user's interests. In addition, the application verifies facts using Wikipedia, stores conversation history, records user feedback, and provides an easy-to-use interface for interacting with all features.

Objective

The objective of this project is to simplify networking at professional events by using Artificial Intelligence to generate relevant conversation starters. The application analyzes event descriptions, extracts the most relevant themes, compares them with the user's interests, and produces engaging discussion topics. It also enables users to verify facts related to event topics and maintains a history of previous conversations and feedback for future reference.

Technologies Used

The project has been developed using Python as the primary programming language. FastAPI is used to build the backend REST APIs, while Streamlit provides an interactive frontend for users. The Hugging Face Transformers library powers the AI-based theme extraction through a zero-shot classification model. Wikipedia is integrated as a reliable source for fact checking, and Pytest is used for automated testing of the application's functionality. Git and GitHub are used for version control and project management throughout the development process.



Project Workflow

The workflow begins when the user enters an event description along with personal interests. The backend receives this information through the FastAPI API. The event description is then processed by the Hugging Face zero-shot classification model, which identifies the most relevant themes. These themes are passed to the conversation generation module, where personalized conversation starters are created by matching the extracted themes with the user's interests.

The user can also verify information through the Fact Check module, which retrieves summaries from Wikipedia. Every conversation generated by the system is stored in a history file, while user feedback is recorded separately. Finally, the generated conversation starters, extracted themes, fact-check results, and history are displayed through the Streamlit frontend.

Project Development Process

The development of the project followed a structured approach beginning with environment setup and architecture planning. Python, FastAPI, Streamlit, and the required AI libraries were installed, and the project structure was organized into separate folders for backend services, frontend, testing, models, and data storage.

The Event Analysis module was developed using the Hugging Face zero-shot classification model. This module analyzes the event description and extracts the three most relevant themes from a predefined set of candidate labels.

Once the themes are identified, the Conversation Generation module creates personalized conversation starters based on the user's interests. These suggestions help users begin meaningful discussions during networking events.

The Fact Check module was implemented to improve the reliability of generated conversations. Whenever a user wants to verify information about a topic, the application retrieves a summary from Wikipedia and presents it through the frontend.

To improve usability, Conversation History and Feedback modules were added. Every generated conversation is stored in a JSON file, allowing users to review previous interactions. Similarly, user feedback is stored separately, enabling future improvements to the recommendation system.

Finally, comprehensive testing was performed using Pytest. Unit tests validated the individual service modules, while API tests ensured that every FastAPI endpoint functioned correctly.

Personalized Networking Assistant

Generate personalized conversation starters for networking events, verify facts, and review your previous interactions.

Generate Conversation Starters

Event Description

AI for Sustainable Cities

Your interests (comma separated)

AI, Machine Learning

 Generate Conversation Starters

Extracted Themes

☒ Artificial Intelligence

☒ Sustainability

☒ Machine Learning

Conversation Starters

"What is your view of the Internet as a new medium for communicating and promoting communication? Where is it going? And what role is it playing? "

 Like

 Dislike

"I want to hear your perspective on how you have seen the technology evolve and evolve, with or without a strong belief that it will continue to evolve. You are right, technology is evolving. It is going to continue to evolve. Your view is that it is going to evolve. You are right, it is going to evolve. Your view is that it is going to evolve."

 Like

 Dislike

"How can this new model of the Internet be used in an environment where people are so much more able to use it? When the old model of the Internet was used, we used a lot of other technology that was used in the '80's to get around stuff. Now we are using technology that will be used. It will continue to be used. It will continue to be used and learn from and learn for the next 5 to 10 years. This is what we are going to make of it."



Fact Check Anything

Enter topic to verify

Blockchain in HealthCare

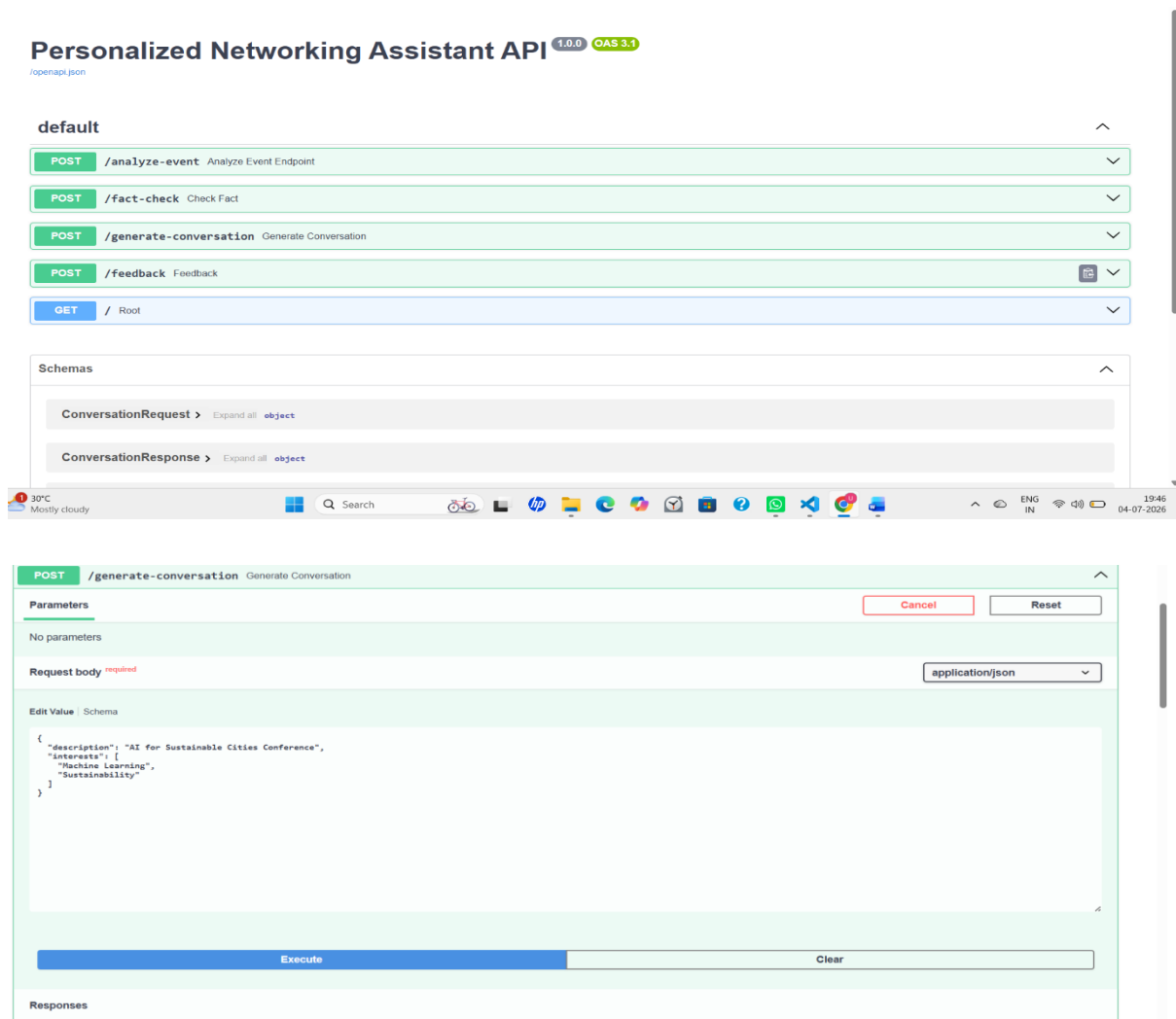
Check Fact

Blockchain privacy describes how blockchain systems handle the confidentiality of transaction and record data. Many public blockchains can provide pseudonymity, while ledger transparency can allow transaction tracing through analysis of on-chain activity and related information. The persistence of ledger data can also create data-protection challenges in some applications, including questions about erasure and rectification rights where personal data are involved, and privacy characteristics vary depending on system design and use case.

API Implementation

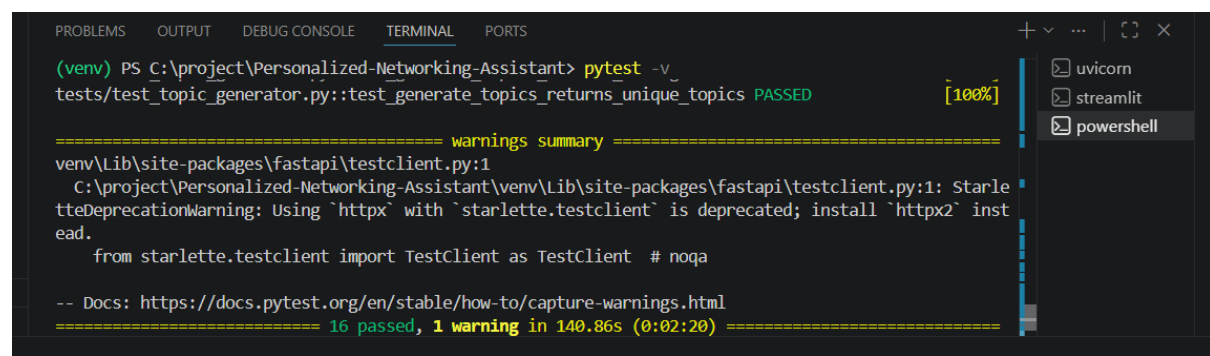
The backend exposes multiple REST APIs to support the application's functionality. The root endpoint confirms that the backend server is running successfully. The Analyze Event endpoint extracts themes from an event description. The Generate Conversation endpoint creates personalized conversation starters using the extracted themes and user interests. The Fact Check endpoint retrieves reliable information from Wikipedia, while the Feedback endpoint records user feedback regarding generated suggestions.

These APIs are documented automatically using FastAPI Swagger UI, making them easy to test and validate during development.



Testing

The project was thoroughly tested using Pytest to ensure the correctness and reliability of every module. Individual unit tests verified the Event Analyzer, Topic Generator, Fact Checker, History Logger, and Feedback Logger. Additional API tests confirmed that each FastAPI endpoint returned the expected responses. All implemented test cases executed successfully without failures, demonstrating the stability of the application.



Results

The completed application successfully analyzes event descriptions, extracts relevant themes using Artificial Intelligence, generates personalized conversation starters, verifies facts using Wikipedia, stores conversation history, records user feedback, and presents all results through a clean Streamlit interface. The backend APIs function correctly, and all automated tests pass successfully. The project demonstrates the practical integration of AI, REST APIs, frontend development, and software testing within a single application.

Challenges Faced

Several challenges were encountered during the development process. Initially, Wikipedia API requests returned HTTP errors that required changes to the implementation. Import issues occurred while configuring the project structure for automated testing, which were resolved by correctly organizing packages and initialization files. Additional debugging was required for feedback logging, API integration, and JSON data handling. These challenges provided valuable experience in debugging, dependency management, API development, and software testing.

Future Enhancements

The project can be extended by integrating additional fact-checking sources, implementing user authentication, replacing JSON storage with a relational database, supporting multilingual conversation generation, deploying the application on cloud platforms, and developing a mobile version for easier accessibility.

Conclusion

The Personalized Networking Assistant successfully demonstrates the application of Artificial Intelligence in solving a real-world networking problem. By combining FastAPI, Streamlit, Hugging Face Transformers, Wikipedia integration, and automated testing, the project provides users with an intelligent platform for discovering event themes, generating meaningful conversation starters, verifying information, and maintaining interaction history. The project not only fulfills its functional requirements but also showcases modern software engineering practices, making it a valuable academic and portfolio project.